HPI Hasso Plattner Institut
Digital Engineering · Universität Potsdam

# Memory & file systems cheat sheet

Practical 04 | Everything runs in the browser Linux; address translation and page replacement are worked out on paper.

## 01 How much memory?

```
free -m                          RAM and swap in MB
grep Swap /proc/meminfo          values from the kernel
```

/proc is written by the kernel the moment you read it. Swap = disk space for pages when RAM is full (the pantry next to the counter). This server: 39 MB RAM, no swap.

## 02 Address spaces

Every process sees its own virtual address space. One region per line: start-end (hex), rights, name.

```
cat /proc/self/maps              your own map
cat /proc/1/maps                 refused: root's process
echo $(( (0xbfbdd000 - 0xbfbbc000) / 4096 ))  33
```

r = read, w = write, x = execute. Code is r-x, the stack rw-. Stack addresses change with every process (randomization), its size does not.

## 03 Pages and page faults

`page = address ÷ 4096` `offset = address mod 4096`

```
echo $(( 30000 / 4096 )) $(( 30000 % 4096 ))   7 1328
```

`physical = frame × 4096 + offset`

Page table: page 7 → frame 2, so 30000 → 2 × 4096 + 1328 = 9520. The offset never changes.

```
/usr/bin/time -v COMMAND 2>&1 | grep -E 'Minor|Page size'
```

| dd, 8 MB buffer | Minor page faults |
|---|---|
| **count=0 (reserved)** | ≈ 40 |
| **count=1 (filled)** | ≈ 2090: + 2048 pages = 8 MB ÷ 4 kB |

**Demand paging:** a page costs RAM only when it is touched for the first time.

## 04 When RAM runs out

```
ps -o pid,vsz,rss,comm           VSZ reserved, RSS in RAM (kB)
grep VmRSS /proc/PID/status      the same for one process
dmesg | grep 'Out of memory'     the kernel's report
```

No swap and RAM full: the OOM killer stops a process, usually the largest. The shell prints Killed, everything else keeps running.

## 05 Page replacement

Requests 1 2 3 4 1 2 5 1 2 3 4 5, all frames empty at the start.

| Strategy | Evicts | misses, 3 frames | 4 frames |
|---|---|---|---|
| **FIFO** | the page that came in first | 9 | 10 |
| **LRU** | the page unused for longest | 10 | 8 |
| **OPT** | the page needed latest (needs the future) | 7 | 6 |

**Belady's anomaly (1969)**

With FIFO, more frames can cause **more** misses. LRU and OPT never get worse with more frames. Real systems approximate LRU (Clock: one reference bit per page).

## 06 Names, inodes and links

An **inode** holds everything about a file except its name: owner, size, times, block addresses. A folder maps names to inode numbers.

```
ls -i FOLDER                     inode numbers
ln FILE NEWNAME                  hard link: same inode
ln -s FILE NEWNAME               symbolic link: a path
ls -l                            2nd column: names
```

rm removes a name. The data is freed when the last name is gone. A symbolic link breaks when its target name disappears.

## 07 Blocks and file systems

```
ls -l FILE                       content size (bytes)
du -k FILE                       space on disk (kB)
df -T                            file systems, types
mount | head -n 1                nr_inodes: max. files
ls FOLDER | wc -l                names in a folder
```

2 bytes occupy one 4 kB block (internal fragmentation). tmpfs keeps files in RAM. Every file needs an inode: a file system can report No space left on device while bytes are free (on a normal Linux: df -i).

| Memory | File system |
|---|---|
| **page / frame** | block / sector |
| **page table** | inode (or FAT) |
| **TLB** | page cache |
| **page fault** | read from disk |